

Reflection on AI-Assisted Development

Throughout my process of building the comment system on the website, using AI was really helpful, honestly way more than I expected. When I first started coding the comment system, I had a rough idea of how PHP forms worked and how to connect to MySQL due to experience in lab 9, but I'd never built a full comment system with form validation, database insertion, and displaying comments. Using AI basically acted like a guide that filled in the gaps I didn't even realize I had.

One example where AI really helped me learn something was when I asked how to structure the comment display. I originally set up all these nested `<div>` wrappers and custom classes, and my CSS looked like a mess. I hence asked AI to reorganize my HTML into simpler blocks and as a result it suggested switching to a single "container" and clearer class names like `.comment` and `.feature`. Seeing the before-and-after made me ultimately understand how unnecessary complexity can make debugging harder. Additionally, a piece of AI-generated code I had to fix was the database query that displayed comments. My generative AI (ChatGPT) gave me a loop that wrapped everything inside an unnecessary `<div class="comments-section">`, which didn't match the new CSS I switched to. It made my layout look broken and I had to strip out the old wrappers and rename classes so they aligned with my CSS. Although this simple error was not hard to fix, it showed me that AI doesn't automatically match my style unless I explicitly tell it to and that the result of what the AI gives me and the quality of the code it suggests all depends on the quality and detail provided in the prompts I give it.

Furthermore, an example of a time that my generative AI of choice gave me code that I did not understand was when it generated a more advanced SQL query with prepared statements. At first, I did not know what `$stmt->bind_param()` did, because I didn't know how it mapped to my columns. Instead of just copy-pasting it, I went and looked up the function and asked the AI to break it down line-by-line. That helped me understand why prepared statements matter and how SQL injection works. I didn't end up using the prepared version right away, but now I actually understand it instead of mindlessly throwing it in. To ensure I understood the AI-generated code, I made myself re-type the code it suggested rather than copying and pasting it. Rewriting it forced me to read every line and made sure I understood how it worked and why the AI suggested implementing it in the code. I also would ask myself if I could explain this to someone else. If the answer was no, I went back and asked AI for clarification until the logic felt natural.

Rather than making me "lazy", AI helped free me up to focus on the harder parts I would've been stuck on for hours. I still had to think through debugging, file structure, SQL schema issues, and how I wanted my comment page to look. AI didn't magically solve the problems, but rather it just helped me get unstuck faster. For example, instead of searching Google for a long time trying to remember how mysqli handles errors, I asked AI and got the

answer in seconds. That let me spend my energy on testing, validating form data, and cleaning up the UI. If AI tools weren't available, I would've eventually gotten everything working, but it would've taken a lot longer. I would've had to dig through StackOverflow and do more internet digging, which is hit or miss, and I might've ended up copy-pasting code I understood less than the AI versions since AI explains things in a way I can ask follow-up questions about and expand further on.

At the end of the day, AI tools are going to change what it means to be a "good programmer." for better. It's less about memorizing syntax and more about being able to read, critique, and adapt code. The skill of understanding logic, debugging, and thinking in abstractions becomes more important. This forces programmers to be able to fully understand the complexity and reasoning behind their code since it is a lot more common to generate functioning code, but still needs programmers to debug and connect the code to real world problems and systems. Hence, I agree with the pedagogical approach of "using AI as a tool" rather than banning it. AI didn't replace learning for me; it sped it up and pushed me to understand things I otherwise might've glossed over.